

**T.C.
ONDOKUZ MAYIS ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



WEB PROGRAMLAMA LABORATUVARI

FÖY – 4

GÜVENLİ YAZILIM GELİŞTİRME ve GÜVENLİK TESTLERİ

6. GÜVENLİK TESTİ YAPABİLME ve GÜVENLİ WEB UYGULAMA GELİŞTİRME

Yazılımın, saldırı veya tehdit altındayken işlevlerini doğru bir şekilde yerine getirmeye devam edecek şekilde korunmasına, yazılım güvenliği denir. Yazılım güvenliği faaliyetlerinin amacı tüm bilgi güvenliği saldırılarına karşı daha dirençli ve hatasız çalışan yazılım üretmektir. Bu föy kapsamında güvenlik testi yapabilme ve güvenli web uygulaması geliştirilmesi üzerinde temel bilgilere yer verilecektir. Bu bilgiler ise şöyle sıralanabilir:

- Güvenli yazılım geliştirme kültürünün oluşturulması
- Yazılım projeleri boyunca kullanılacak süreçleri tanımlarken güvenli yazılım geliştirme kuralların nasıl ve ne zaman uygulanacağını belirlenmesi
- Yazılımların siber güvenlik tasarımının yapılması ve güvenlik tasarım dokümanını oluşturulması
- Yazılım tasarımının siber güvenlik gözü ile gözden geçirilmesi
- Yazılım güvenlik testlerinin gerçekleştirilmesi
- Güvenli yazılım geliştirme yaşam döngüsü faaliyetlerinin izlenmesi ve denetimi
- Yazılım güvenliğinin sürekliliğinin sağlanması

6.1. GÜVENLİ YAZILIM GELİŞTİRME

Yazılıma karşı yapılan saldırıların üç ana amacı bulunmaktadır:

- Yazılımın çalışmasını tamamen durdurmak ya da doğru bir şekilde çalışmasını engellemek,
- İçine zararlı kod dâhil ederek veya yazılımın kodunu değiştirmek yoluyla, yazılımın çalışma akışını değiştirerek istemeyen ya da öngörülmeleyen bir amaca hizmet etmesini sağlamak,
- Yazılımın çalışmasındaki işlemler ve çevresiyle ilgili bilgi edinerek yazılımın zayıf noktalarını tespit etmek, çalışma ortamına sızmak ve istenilmeyen amaçları gerçekleştirmek için aracı olarak kullanmak

Yazılıma yapılan saldırıların önlenmesi için öncelikle yazılımların, yazılım güvenliği ilkelerine uygun olarak tasarlanması ve geliştirilmesi, yazılım güvenliğini sağlayacak yöntemlerin ve araçların kullanılması gerekmektedir. Bütün bu çalışmalar da bütüncül bakış açısı ile güvenli yazılım geliştirme yaşam döngüsü içinde gerçekleştirilmesi ve bu konudaki olgunluğun geliştirilmesi gerekmektedir. Bu bağlamda aşağıdaki bölümlerde sırası ile yazılım güvenliği

ilkeleri, yazılım güvenliğini sağlayan yöntemler, güvenli yazılım geliştirme yaşam döngüsü, güvenlik test araçları açıklanmıştır.

6.1.1. YAZILIM GÜVENLİĞİ İLKELERİ

Yazılım güvenliği ilkeleri güvenlikle ilgili tüm mekanizmaların tasarımında kullanmak gereklidir. Teknik hususların yanı sıra insan etkileşimiyle ilgili hususların da göz önüne alması önemlidir. Teknik ve insan etkileşimlerini de kapsayacak şekilde, güvenli yazılım geliştirme yaşam döngüsünde dikkate alınması gereken temel yazılım güvenliği ilkeleri şöyledir:

Gerek Duyulan En Az Yetkiyi Ver: En az yetki ilkesi, her bir yazılım prosedürünün kendi işini bitirmesi için gerekli en az hak kümesiyle çalıştırılmasıdır. Bir öğeye (metot, kullanıcı, yazılım vb.) belirli bir görevi tamamlaması gereken en az yetki kümesi sağlanmalıdır. En az yetki ilkesinin yararları arasında şunlar sayılabilir:

- Sistem kararlılığına olumlu etki: uygulamanın sistem üzerinde yapabileceği değişiklikler sınırlanır
- Sistem güvenliğine olumlu etki: bir uygulamadaki zayıflıkların sistemin geri kalanına sızmak için kullanılamaz
- Konuşlandırma kolaylığı: bir uygulamanın daha az yetki gerektirmesi karmaşık bir ortama konuşlandırılmasını kolaylaştırır.
- En az yetki ilkesi kapsamında ayrıca aşağıdaki hususlara da dikkat edilmesi gerekmektedir.
- Sistem kaynaklarına erişmesi gereken uygulamalara yalnızca erişmesi gereken kısıtlı bölgeler için yetki verilmeli ve yönetici haklarıyla çalıştırılmamalıdır.
- Yükseltilmiş hakları gerektiren yazılım süreçleri bu haklara sadece gereken en az süre boyunca sahip olmalıdır.

Tüm Erişimleri Denetle: Her bir nesneye yapılan her bir erişimde yetki kontrolü yapılmalıdır. Bu ilkeye uyum, sistem genelinde erişim denetimi sağlanmasını gerektirir. Tüm erişimleri denetle ilkesi kapsamında ayrıca aşağıdaki hususlara da dikkat edilmesi gerekmektedir.

Yetkileri Ayır: Bir sistem yüksek güvenli bir işleme yalnızca tek bir koşula bağlı olarak izin vermemelidir. Yüksek güvenli bir işleme izin verilmesi için birden fazla koşulun sağlandığı doğrulanmalıdır.

Varsayılan Değerleri Güvenli Hale Getir: Kullanılan tüm varsayılan değerlerin güvenliği artıracak şekilde seçilmesi gerekmektedir. Bu konuda sık görülen hatalar şunlardır:

- Çalıştırılabilir dosyaların varsayılan olarak herkesin yazabileceği şekilde kurulması
- Uygulama ana dizininin herkesin okuyabileceği şekilde kurulması
- Herkesin yazabileceği iz kaydı dosyaları
- Herkesin okuyabileceği dosyalarda varsayılan (geliştirme, test) parolaların bulunması
- Herkes tarafından okunabilen dizinler
- Cihazlarda IP sahtekarlığına izin veren varsayılan ayarlar kullanılması
- Paylaşılan anahtar dosya / veri tabanlarında güvensiz hakların verilmiş olması

Bu gibi açıklıklara önlem olarak tüm varsayılan değerlerin gözden geçirilmesi ve güvenli değerlerin atanması gereklidir. Varsayılan değerleri güvenli hale getirilmesi kapsamında ayrıca aşağıdaki hususlara da dikkat edilmesi gerekmektedir.

- Bir öğeye özellikle bir nesne üzerinde erişim verilmemiş ise bu öğeye erişimi olmamalıdır.
- Bir nesnenin varsayılan erişimi hiç olmalıdır.
- Parola süresinin dolması, karmaşıklık kontrolü gibi kontroller varsayılan durum olmalıdır.
- Kriptografik algoritmaların çalışma kümesi varsayılan olarak en güvenli küme olmalıdır.
- Kimlik denetiminde en güvenli faktör varsayılan olarak kullanılmalıdır.

Ortak Erişilen Kaynaklara Farklı Kanallardan Eriş: Kaynaklara erişen mekanizmalar ortak kullanılmamalıdır (örneğin paylaşılan dosyalar). Ortak kanallardaki bilgi akışından kaynaklanan bilgi sızması/bozulması problemleri ve yan kanal saldırıları gibi problemlere neden olmaktadır. Ayrıca sistem ve yazılım içerisinde farklı bileşenler arasında da yalıtım sağlanmalıdır.

En Zayıf Halkayı Tespit Et ve Güçlendir: Bir yazılımın ilk saldırıya uğrayacak noktası, yazılımın en zayıf noktasıdır. Yazılımın güvenliği en zayıf halkanın güvenliği kadardır. Güvenlik analizi adımıyla en zayıf halka tespit edilmelidir. Bu halka kendisine yönelik riski karşılayabilecek şekilde yeniden tasarlanıp gerçekleştirilmelidir.

Saldırı Yüzey Alanını Azalt: Saldırı yüzey alanı, bir yazılım için yetkisiz bir kullanıcının (saldırgan) yazılım ortamından veri alabileceği, veri girebileceği veya yetkisiz işlem yapabileceği noktaların tümüdür. Uygulamaya gereksiz özellikler eklenmemelidir. İletişim

yapılacak noktalar sınırlanmalıdır. Saldırı yüzeyi tasarım aşamasında belirlenmeli ve sürekli olarak izlenmelidir.

Savunma Derinliği Oluştur: Savunma derinliği, savunma mekanizmalarının peş peşe uygulanmasından oluşur. Bu şekilde bir savunma mekanizması aşıldığında, sistem tamamen savunmasız kalmaz. Bu bağlamda aşağıdaki hususlar dikkate alınmalıdır:

- Farklı üreticilerin/geliştiricilerin işletim sistemi, yazılım kütüphaneleri, güvenlik yazılımlarının kullanılması
- Yazılımdaki güvenlik mekanizmalarının, sistemdeki güvenlik mekanizmalarıyla entegrasyonu
- Güvenlik politikalarının tüm bileşenlerde dikkate alınması ve zorunlu olarak uygulanmasının sağlanması

Basit Güvenlik Mekanizması Tasarla: Güvenlik mekanizmaları mümkün olduğunca basit olmalıdır. Bu şekilde hata olasılığının daha az olması sağlanır, hatalar oluştuğunda anlaşılması ve düzeltilmesi kolaylaşır. Bu kapsamda özellikle arayüzlerin mümkün olduğunca basit ve dokümantasyonunu yapılmış olması gereklidir.

Anlaşılabilir ve Kolay Kullanılabilir Güvenlik Mekanizması Tasarla: Yazılımda kullanıcıyla etkileşim gerektiren güvenlik mekanizmaları anlaşılabilir ve kolay kullanılabilir olmalıdır. Aksi takdirde kullanıcı güvenlik mekanizmalarının etrafından dolaşmak için yollar arayabilir. Bu bağlamda aşağıdaki hususlar dikkate alınmalıdır:

- Kullanıcı işini en kolay şekilde ancak en az yetkiyle yapabilmelidir.
- Kullanıcı yapacağı işe ilişkin güvenlik politikalarını tanımlayabilmelidir.
- Kullanıcının izin verdiğini gösteren eylemleriyle yetkilendirme otomatik olarak verilebilmelidir.
- Kullanıcı daha önce verdiği yetkileri geri alabilmelidir.
- Kullanıcı verdiği yetkileri görebilmelidir.

6.1.2. YAZILIM GÜVENLİĞİNİ SAĞLAMA YÖNTEMLERİ

Yazılımdaki hataları azaltmak ve kalitesiyle güvenliğini artırmak için birçok teknik yöntem mevcuttur. Bu bölümde beş ana başlık altında, güvenlik odaklı olarak kullanılabilecek temel yöntemler açıklanmıştır.

Sezgisel (Heuristic) ve Benzetime (Simulation) Dayalı Yöntemler: Bu tür yöntemlerin arasında yer alan statik analiz ve sembolik koşturma yöntemlerinde, yazılımın çalıştırılan halini yansıtan bir modeli veya benzetimi oluşturulur ve bilinen yazılım hata türleri bu model üzerinde aranır. Dinamik analiz, negatif ve rastgele test yöntemlerinde ise doğrudan yazılımın koşturulması sırasında gerçek ortamın benzetimi oluşturularak yazılımın bazı hatalara zorlanması sağlanır. Oluşan hatalar analiz edilir ve geliştirilmiş sonuçlara ulaşılmaya çalışılır.

Statik Analiz (Static Analysis): Statik analiz, bir yazılımın belli başlı özelliklerini onu çalıştırmadan analiz etmeyi amaçlayan bir yöntemdir. Sezgisel (heuristic) yöntemler, güvenilir (sound) analizlere göre daha hızlıdır ancak mantıksal tümevarım yöntemlerinin sahip olduğu güvenceyi sağlayamazlar. Yazılımın birçok gösterimi statik analize tabi tutulabilir (gereksinim, mimari, kaynak kodu, çalıştırılabilir dosya gibi) ancak bunların arasından en olgun yöntemler kaynak kodu statik analiz yöntemleridir. Statik analiz yazılımlarını karşılaştırırken hangi tür yazılım hatalarını çözebildikleri harici kaynaklardan doğrulanmalıdır.

Sembolik Koşturma (Symbolic Execution): Statik analizle yazılımdaki bazı hataların bulunması mümkün olmaktadır. Ancak statik analizlerle kodun derinlerde gizli hataların bulunması çoğu zaman mümkün olmamaktadır. Bunun nedenlerinden biri statik analiz ürünlerinin yanlış alarm oranını azaltmak ve analizi basitleştirerek bazı tür hataları analiz kapsamı dışında bırakmasıdır. Sembolik koşturma yönteminde yazılımın kodu üzerinde bir soyutlama yapılarak tüm olası çalıştırmalar modellenmektedir. Yazılım testi kullanılarak sadece tanımlanan akış kapsamındaki hatalar bulunabilir. Sembolik koşturma yöntemi, test yöntemini geliştirmekte ve daha fazla akış analiz etmektedir. Sembolik koşturmada bir sembolik koşturma ağacı oluşturulur. Bu ağaç üzerindeki her bir yol birçok gerçek program çalışma akışına eşdeğerdir. Her bir yol sanki program koşturuluyormuş gibi işletilir. Bu sayede programı koşturmadan yazılım testi yöntemine göre daha fazla kapsama sağlanabilmektedir.

Dinamik Analiz (Dynamic Analysis): Dinamik analiz, uygulamadaki açıklıkları tespit etmek için çalışan yazılım üzerinde ve çalışma ortamında gerçekleştirilen analiz araç ve yöntemleridir. Dinamik analiz araçları ve yöntemleri şu şekilde özetlenebilir:

- **Ağ tarayıcısı:** Ağ üzerindeki cihazlar ile yazılımları belirleyen ve bunların özelliklerini tespit eden araçlardır. Örnek: nmap
- **Ağ dinleyicisi:** Ağ üzerindeki trafiği dinleyerek anlık analiz eden veya sonradan analiz edilmek üzere kaydeden araçlardır. Örnek: wireshark

- **Ağ zayıflık tarayıcısı:** Bir ağ cihazına, yazılımına veya ağ üzerindeki bir servise daha önceden oluşturulmuş ağ trafiği göndererek güvenlik politikalarına uygunluğunu veya bilinen zayıflıkları tespit etmekte kullanılan araçlardır. Örnek: OpenVAS
- **Bilgisayar tabanlı zayıflık tarayıcısı:** Kurulum ve işletim sırasındaki yazılım ile ilgili güvenlik açıklıklarını tespit etmek için bilgisayar üzerindeki yapılandırmaları ve denetim araçlarını test eden araçlardır. Örnek: OpenSCAP
- **Uygulama zayıflık tarayıcısı:** Belirli bir tür uygulamaya girdi üreterek bilinen veya bilinmeyen zayıflıkları tespit etmeye yarayan araçlardır. Örnek: Web uygulaması/servisi zayıflık tarayıcısı (OWASP ZAP, Nikto, Vega vb.), Veritabanı zayıflık tarayıcısı (sqlmap, Scuba)

Negatif Test (Negatif Testing)

Hatalı veya güvenlik kurallarına uymayan durumların ve istisnaların yazılım tarafından kontrol edildiğinin test edilmesine negatif test yöntemi denir. Örneğin; en az 8 karakterlik bir parola gerektiren bir sayfanın 7 karakterli parola ile test edilmesi ve sonuçta yazılımın bu hususta hata mesajı vermesi.

Rastgele Test

Bir yazılıma geçersiz, beklenmeyen veya rastgele veri girdisinde bulunarak istenmeyen davranışta bulunup bulunmayacağının tespit edilmesine rastgele test yöntemi denir. Birçok uygulama zayıflık tarayıcısı aynı zamanda rastgele test kabiliyetlerine de sahiptir.

6.2. GÜVENLİK TEST ARAÇLARI

6.2.1. Kaynak Kod Analizi Test Araçları

Statik analiz ile uygulama çalıştırılmaksızın, uygulamanın kaynak kodları üzerinden analiz yapılmaktadır. Bu analiz gözden geçirme şeklinde olabileceği gibi otomatik araçlar aracılığıyla olabilir. Otomatik araçlar ile hem kod kalitesi ölçülebilmekte hem de güvenlik hatalarına sebebiyet verebilecek kodlama hataları tespit edilebilmektedir. Kod kalitesi ölçümünde pep8 vb. kodlama stilleri üzerinden kontroller yapılır. Bu kontroller kodlamanın yazım biçimlerini, kod içinde bulunan boşluklar, değişken isimleri, fonksiyonların girdi sayılarını, satır sayılarını ve benzeri özellikleri kontrol eder. Otomatik araçlar aracılığıyla, güvenlik açıklıklarına sebebiyet verebilecek kodlama hataları da tespit edilebilmektedir. Örneğin boyut kontrolü yapılmadan kopyalanan bir karakter katarının uygulama için taşma hatası yaratabileceği yerler otomatik olarak tespit edilebilmektedir. Otomatik araçlar ürettiği hata çıktılarının bir kısmı

yanlış alarm (false pozitif) hatalardır. Bunlar gerçekte güvenlik açığı oluşturmadığı halde statik analiz araçları tarafından tespit edilen hatalardır. Yanlış alarm hataların tespit edilmesi için otomatik araçların çıktıları yine manuel olarak gözden geçirilmelidir. Statik kod analizde kullanılabilecek örnek araçlar Tablo 1’de listelenmiştir.

Tablo 1. Kaynak Kod Analizi Test Araçları

Araç Adı	Erişim linki
Boon	http://www.cs.berkeley.edu/~daw/boon
Find Security Bugs	http://h3xstream.github.io/find-sec-bugs/
FindBugs	http://findbugs.sourceforge.net
FlawFinder	http://www.dwheeler.com/flawfinder
FxCop	https://www.microsoft.com/en-us/download/details.aspx?id=6544
Google CodeSearchDiggity	http://www.bishopfox.com/resources/tools/google-hackingdiggity/attack-tools/
Oedipus	http://www.darknet.org.uk/2006/06/oedipus-open-source-webapplication-security-analysis/
phpcs-security-audit	https://github.com/Pheromone/phpcs-security-audit
PMD	http://pmd.sourceforge.net/
SonarQube	http://sonarqube.org

6.2.2. Sızma Testi Araçları

Sızma testlerinde yöntem olarak kara kutu, gri kutu ve beyaz kutu yaklaşımları kullanılmaktadır. Beyaz kutu yaklaşımında kaynak kod dâhil sistem hakkında tüm bilgilere sahipken, gri kutu yaklaşımında ise kapsam hakkında sınırlı bilgi ile sızma testi gerçekleştirilir. Kara kutu yaklaşımında test edilen sistem hakkında hiçbir ön bilgi bulunmadığı kabul edilmektedir. Testi gerçekleştiren ekibin kaynak koda erişimi bulunmamaktadır. Sisteme yapılan girdiler karşısında sistemin verdiği çıktılar gözlemlenerek sistem hakkında bilgiler edinilip, elde edilen bilgiler ışığında yeni testler yapılmaktadır. Sızma testleri aşağıdaki adımları içermektedir:

Keşif: Pasif ve aktif bilgi toplama aşamalarından oluşur. Pasif bilgi toplamada test edilen sistemle etkileşime girilmeden, çevrimiçi kaynaklar kullanılarak hedef sistem ile ilgili bilgi toplanır. Aktif bilgi toplama aşamasında ise test edilen sistem ile etkileşime girilerek bilgi toplamaya çalışılır. Toplanan bilgiler uygulamanın mimarisi, kullandığı teknolojiler, sürüm bilgileri, servis bilgileri, IP bilgileri, port bilgileri, yapılandırma ayarları, geliştirici bilgileri, kullanıcı verileri gibi çeşitli bilgilerden oluşur. Keşif ve sızma testlerinde kullanılabilecek örnek araçlar Tablo 2’de listelenmiştir.

Tablo 2. Keşif ve Sızma Testi Araçları

Araç Adı	Erişim linki
Aircrack-ng	https://www.aircrack-ng.org/
Cain & Abel	http://www.oxid.it/cain.html
Hping	http://www.hping.org/
THC-Hydra	http://www.thc.org/thc-hydra/
Immunity Canvas	http://www.immunityinc.com/products/canvas/
John the Ripper	http://www.openwall.com/john/
Metasploit Framework	https://www.metasploit.com/
Nmap	https://nmap.org/
p0f	http://lcamtuf.coredump.cx/p0f3/

Zafiyet Tarama: Bu aşamada keşif aşamasında bulunan bilgilerden faydalanılarak uygulama bileşenlerinin bilinen açıklıkları belirlenir. Keşif aşamasında elde edilen bulgularda hata olabileceği veya uygulama bileşenlerinde yama uygulanmış olabileceği için açıklığın gerçekten var olup olmadığının tespiti için zafiyetin sömürülmeye çalışılması gerekmektedir. Zafiyet taramada kullanılabilecek örnek araçlar Tablo 3’de listelenmiştir.

Tablo 3. Web Sızma Testi Araçları

Araç Adı	Erişim linki
Burp Suite	http://www.portswigger.net/Burp/
Metasploit	https://www.metasploit.com/
Nikto	https://cirt.net/nikto2
OWASP Mantra	http://www.getmantra.com/

SPIKE	http://www.immunitysec.com/resources-freesoftware.shtml
W3af	http://w3af.sourceforge.net/
Vega	https://subgraph.com/vega/

6.2.3. Rastgele Test Araçları

Rastgele testi ile uygulamanın girdi noktalarına beklenmedik rasgele veriler sistematik olarak gönderilerek uygulamanın ürettiği hatalar gözlemlenir. Öncelikle hedef yazılım üzerinde testi gerçekleştirecek girdi noktalarının tespiti edilir. Uygulamamızın girdisi bir dosya formatı (docx, pdf, jpeg, ...), bir protokole ait bir mesaj (ftp, http, arp vb.), çevresel değişken (environment variable), kayıt anahtarı girdisi (register key) ya da kullanıcı tarafından girilen bir form değişkeni şeklinde olabilir. Sonrasında girdi noktalarına gönderilecek verinin formatına uygun veri kümesi oluşturulur, bu kümedeki veriler ya sırayla sistematik bir şekilde ya da rasgele örnekleme yöntemiyle belli bir sayıda uygulamada kullanılır. Rastgele testlerin uygulama girdilerinin herhangi bir hataya neden olup olmadığı sistem ve uygulama iz kayıtlarından gözlemlenir. Son aşamada hata oluşturan girdiler tespit edilerek bulunan hataların güvenlik açısından sömürülebilir (exploitable) olup olmadığı tespit edilir. Rastgele testler için kullanılabilecek örnek araçlar Tablo 4’de listelenmiştir.

Tablo 4. Rastgele Test Araçları

Araç Adı	Erişim linki
OWASP WSFuzzer	https://www.owasp.org/index.php/Category:OWASP_WSFuzzer_Project
Wfuzz	http://www.darknet.org.uk/2007/07/wfuzz-a-tool-forbruteforcingfuzzing-web-applications/

KAYNAKÇA

https://egitim.sge.gov.tr/pluginfile.php/6115/mod_page/content/20/SGE-KLV-GuvenliYazilimGelistirmeKilavuzu_Rev1.0.pdf Son erişim tarihi: 30.09.2018

Sorular

- 1) Zafiyet testi ve sızma testinin farkları nelerdir?
- 2) **Nmap** aracı ile yerel ağ üzerinde şu işlemleri gerçekleştiriniz:
 - a. Ağda bulunan bilgisayarların keşfini gerçekleştiriniz. Basit bir şekil ve tanımlamalarıyla bu ağı gösteriniz (5-6 tane yeterlidir).
 - b. Bilgisayarınızda çalışan servislerin kullandığı port numaralarını bularak bu servislerin ne işe yaradıklarını anlatınız. Bilgisayarınızda herhangi bir port kullanan servis açık değilse (örneğin SSH) en az 2 servis açmaya çalışınız ve bu servisleri anlatınız.
 - c. Tek bir IP (yerel makine IP adresi) kullanarak port taraması yapınız ve elde edilen sonuçları düzgün bir şekilde yorumlayınız.
 - d. Yerel makine üzerindeki 1 ve 1000 nolu portlar arasındaki bütün portları tarayan komutu yazınız ve çıkan sonuçları yorumlayınız.
 - e. TCP bağlantı taraması nedir? TCP bağlantı taraması gerçekleştiriniz.
- 3) **Vega** aracı ile localhost üzerinde çalışan bir web uygulamasına zafiyet testi gerçekleştiriniz ve çıkan sonuçları yorumlayınız. Çıkan hatalardan bir tanesini nasıl kullanabileceğiniz ile ilgili bir örnek veriniz (Kullanacağınız yerel uygulama önceki föylerde yaptığınız uygulamalardan birisi olabilir).
- 4) Kara-kutu ve beyaz-kutu test yöntemleri nedir? Birbirlerinden farklılıkları nelerdir?
- 5) (Bonus) Shodan.io aracını araştırınız. Örnek bir kullanım senaryosu gösteriniz. Elde ettiğiniz sonuçları yorumlayınız.

İncelenebilecek kaynaklar:

- 1) <https://www.slideshare.net/bgasecurity/nmap-kullanm-kitap-27487242>
- 2) <https://www.ozztech.net/siber-guvenlik/nmap-nasil-kullanilir/>
- 3) <https://siberbulten.com/teknik/ag-tarama-araci-nmap-nedir-nasil-kullanilir/>
- 4) <https://pentest.com.tr/blog/Zafiyet-Tarama-Araclari-VEGA-ve-NIKTO.html>
- 5) <https://www.ssl.com.tr/nas%C4%B1l/linux-alt-sistemini-etkinle%C5%9Ftir-ubuntu-windows-10%27u-y%C3%BCkle/>

NOT: Föydeki bazı sorular için Linux işletim sistemini kullanmak daha uygun olabilir. Bunun için sanal makineye işletim sistemi kurabilirsiniz veya Windows üzerinden Ubuntu Shell kullanımı yapabilirsiniz. Bunun için 5 numaralı kaynağı inceleyebilirsiniz.

NOT: Baştan savma ve sadece resim içeren raporlar kesinlikle değerlendirilmeyecektir. Bütün soruların cevaplarını açık ve net bir şekilde ifade etmeye çalışınız.